



UNIVERSIDADE FEDERAL DO PARANÁ – UFPR

Departamento de Informática

Computer Vision and Perception

Camera Model and Calibration

Prof. Eduardo Todt
2026

Introduction

Intrinsic parameters

- Camera geometry
- Optical properties

Extrinsic parameters

- position and orientation of the camera coordinate frame relative to the world coordinate frame

Introduction

Recover 3D information from 2D image coordinates

Project 3D information onto 2D image coordinates

Camera Calibration

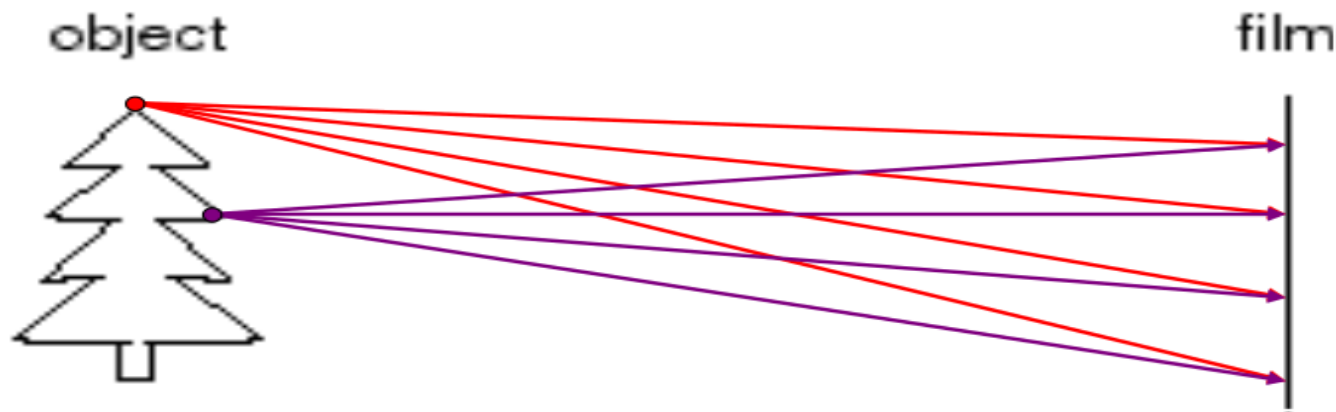
Zhengyou Zhang

<http://research.microsoft.com/en-us/um/people/zhang/>

The following sequence of slides is based on those available on the aforementioned Zhang webpage, with some additions.



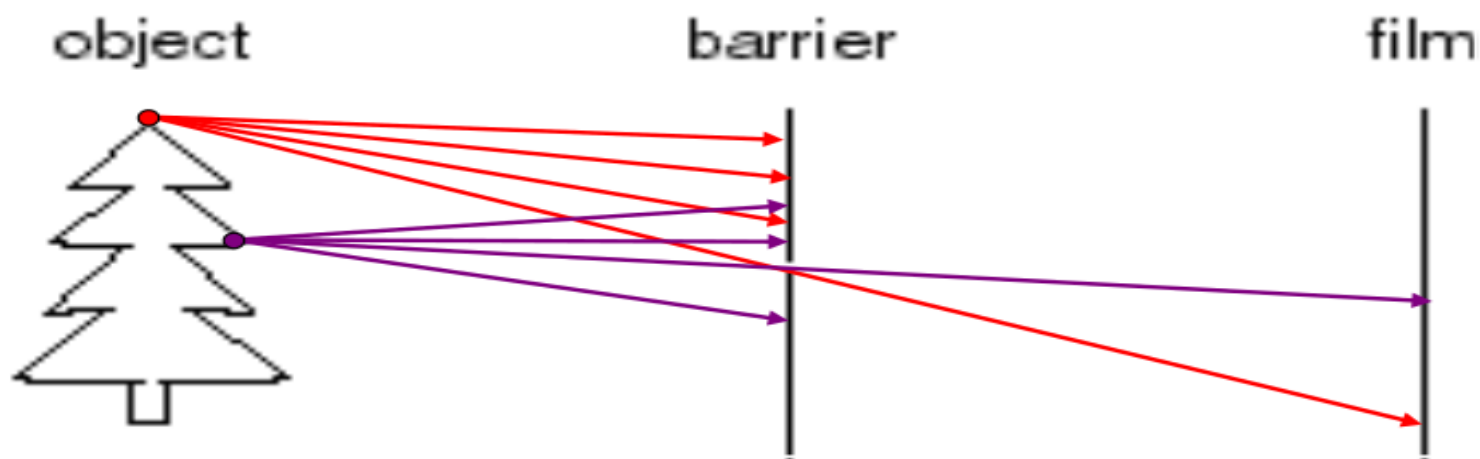
Image formation



Let's design a camera

- Idea 1: put a piece of film in front of an object
- Do we get a reasonable image?

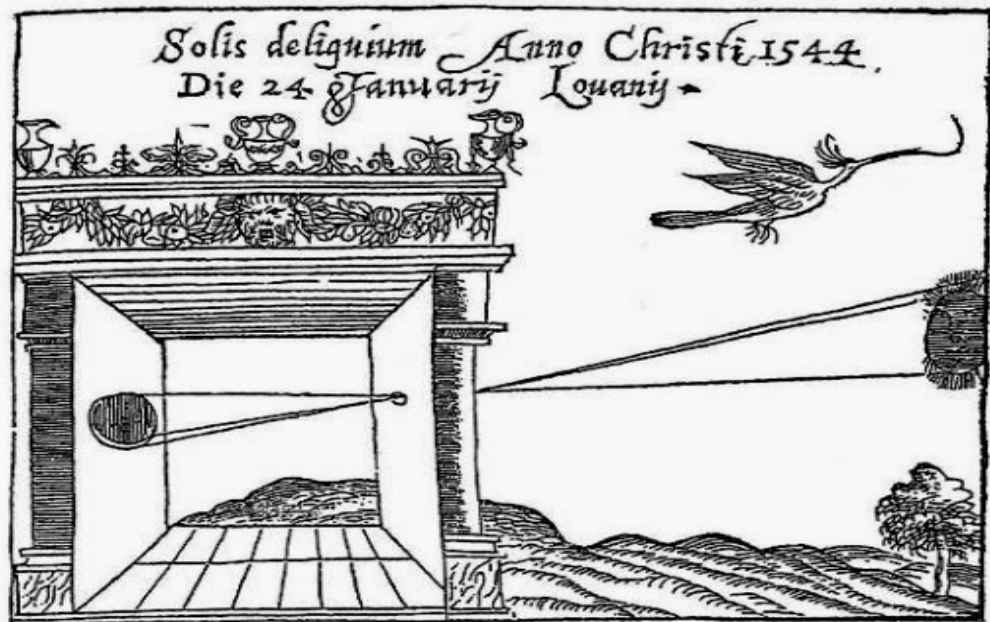
Pinhole camera



Add a barrier to block off most of the rays

- This reduces blurring
- The opening known as the **aperture**
- How does this transform the image?

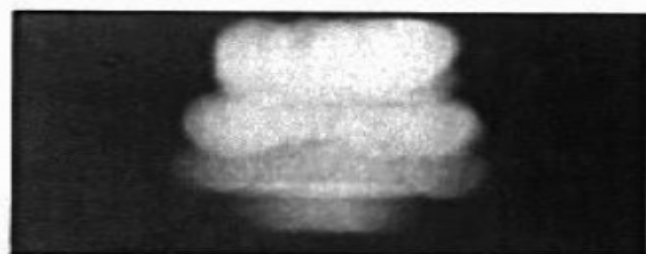
Camera Obscura



Gemma Frisius, 1558

- Basic principle known to Mozi (470-390 BC), Aristotle (384-322 BC)
- Drawing aid for artists: described by Leonardo da Vinci (1452-1519)

Shrinking the aperture



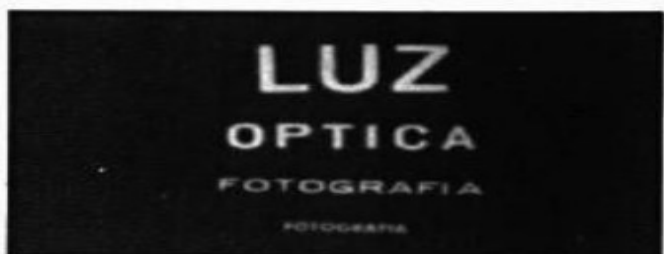
2 mm



1 mm



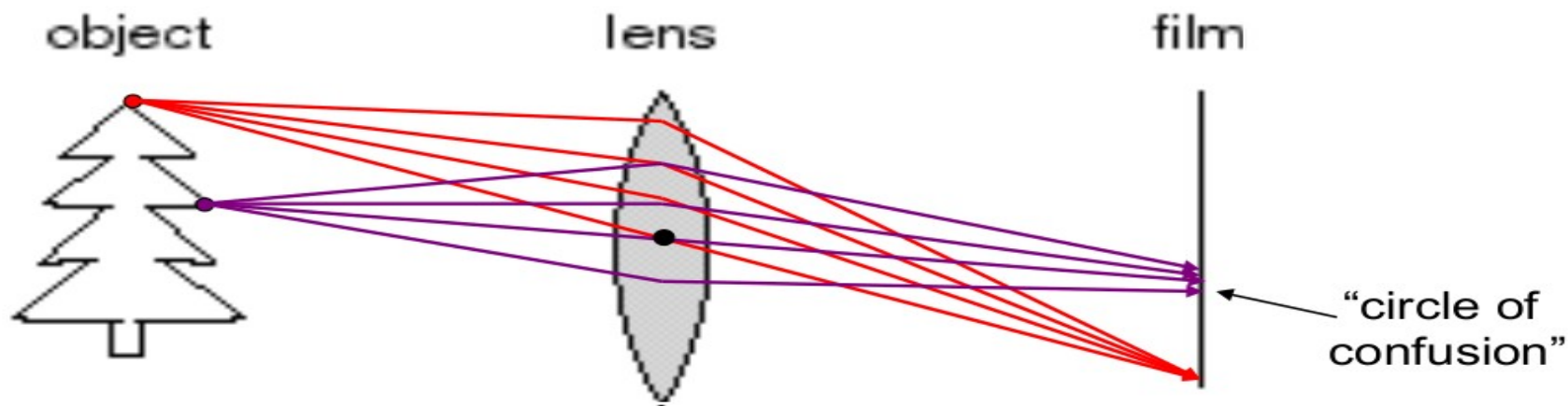
0.6 mm



0.35 mm

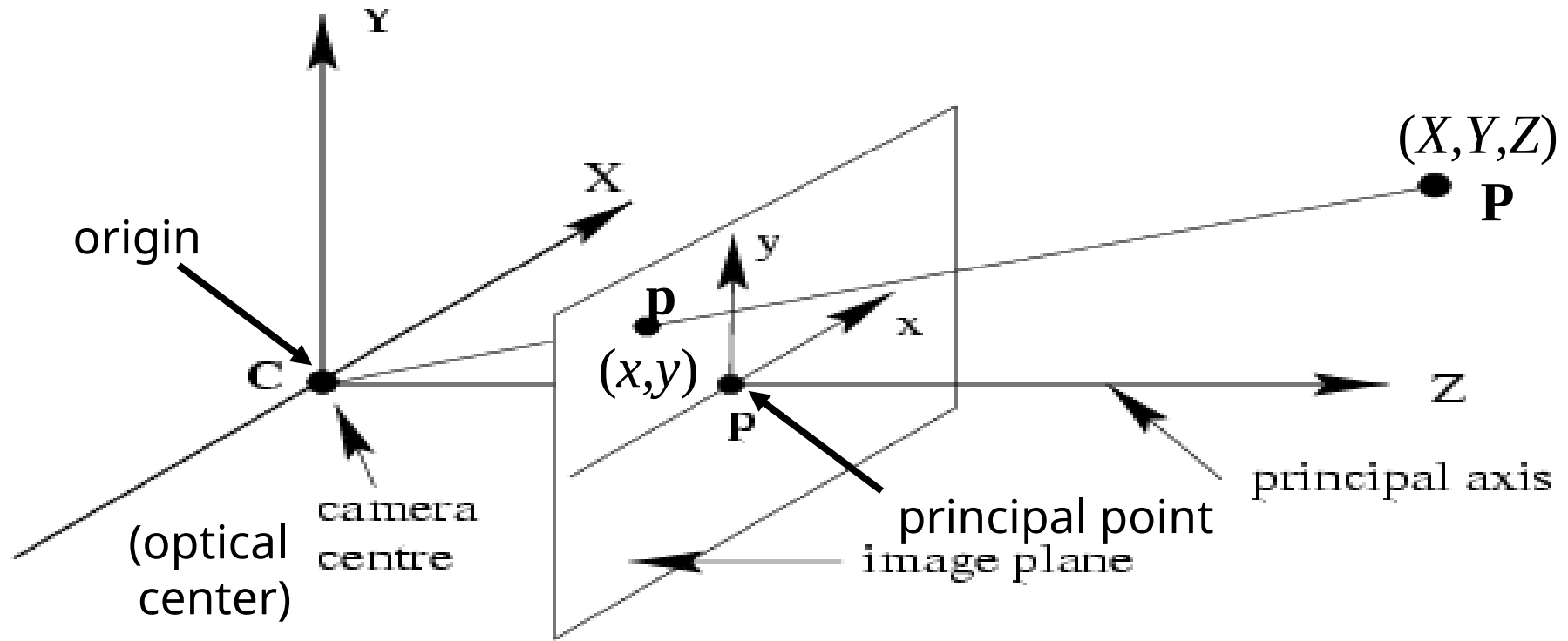
- Why not make the aperture as small as possible?
 - Less light gets through
 - *Diffraction* effects...

Adding a lens



- A lens focuses light onto the film
 - There is a specific distance at which objects are “in focus”
 - other points project to a “circle of confusion” in the image
 - Changing the shape of the lens changes this distance

Pinhole camera model



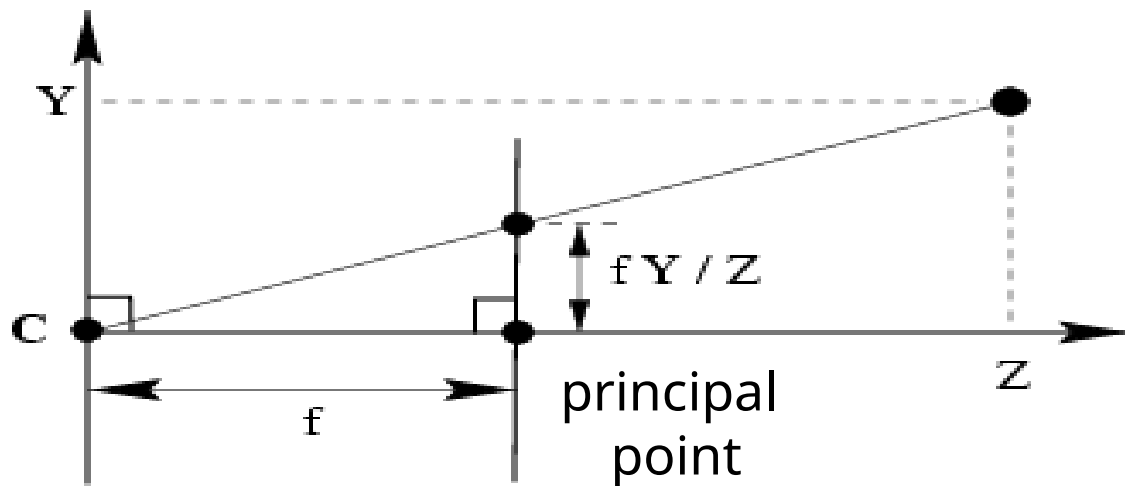
The coordinate system

We will use the pin-hole model as an approximation

Put the optical center (**C**enter **O**f **P**rojection) at the origin

Put the image plane (**P**rojection **P**lane) *in front* of the COP (Why?)

Pinhole camera model

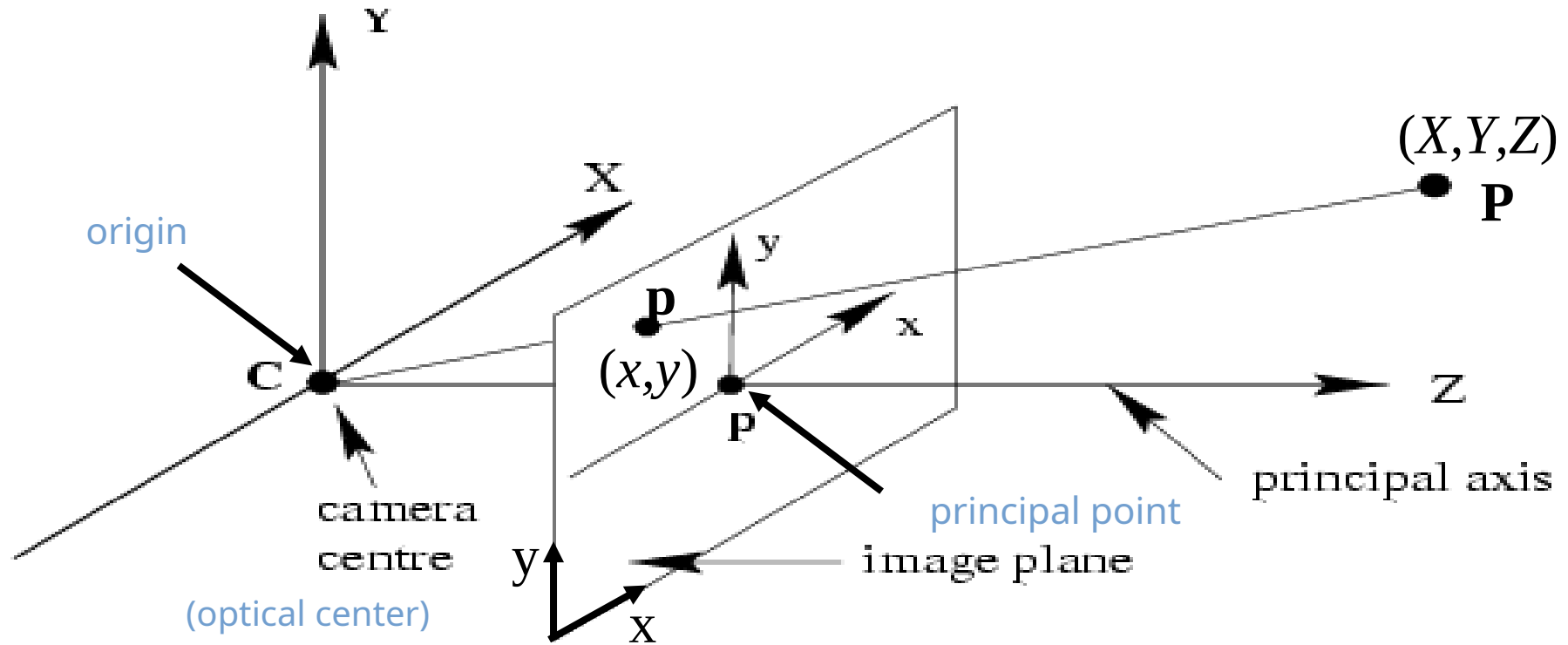


$$x = \frac{fX}{Z}$$

$$y = \frac{fY}{Z}$$

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \sim \begin{pmatrix} fX \\ fY \\ Z \end{pmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}$$

Pinhole camera model



$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \sim \begin{pmatrix} fX \\ fY \\ Z \end{pmatrix} = \begin{bmatrix} f & 0 & x_0 \\ 0 & f & y_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}$$

Intrinsic matrix

Is this form of K good enough?

$$K = \begin{bmatrix} f & 0 & x_c \\ 0 & f & y_c \\ 0 & 0 & 1 \end{bmatrix}$$

non-square pixels:

$$K = \begin{bmatrix} f_x & 0 & x_c \\ 0 & f_y & y_c \\ 0 & 0 & 1 \end{bmatrix}$$

Intrinsic matrix

Is this form of K good enough?

$$K = \begin{bmatrix} f & 0 & x_c \\ 0 & f & y_c \\ 0 & 0 & 1 \end{bmatrix}$$

non-square pixels

skew

$$K = \begin{bmatrix} f_x & s & x_c \\ 0 & f_y & y_c \\ 0 & 0 & 1 \end{bmatrix}$$

Intrinsic matrix

Is this form of K good enough?

$$K = \begin{bmatrix} f & 0 & x_c \\ 0 & f & y_c \\ 0 & 0 & 1 \end{bmatrix}$$

non-square pixels

skew

radial distortion

$$K = \begin{bmatrix} f_x & s & x_c \\ 0 & f_y & y_c \\ 0 & 0 & 1 \end{bmatrix}$$

Distortion

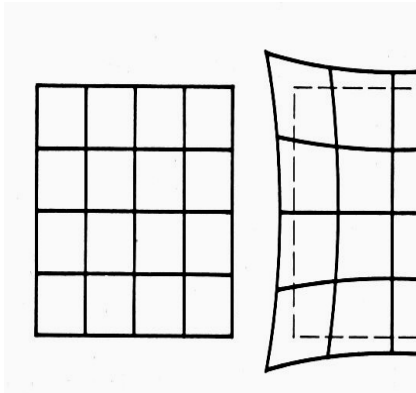


Radial distortion of the image

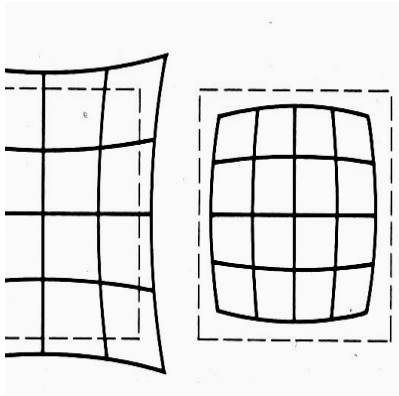
Caused by imperfect lenses

Deviations noticeable for rays that pass through the edge of the lens

Barrel Distortion



No distortion

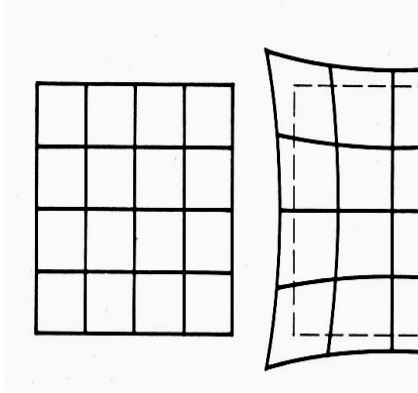


Barrel

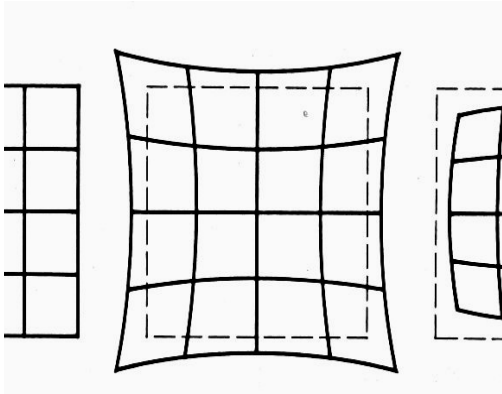


Wide Angle Lens

Pin Cushion Distortion



No distortion



Pin cushion



Telephoto lens

Modeling distortion

Distortion-Free:

$$x = \frac{fX}{Z}$$

$$y = \frac{fY}{Z}$$

With Distortion:

1. Project (X, Y, Z)
to “normalized”
image coordinates

$$x_n = \frac{X}{Z}$$

$$y_n = \frac{Y}{Z}$$

$$r^2 = x_n^2 + y_n^2$$

2. Apply radial distortion

$$x_d = x_n \left(1 + \kappa_1 r^2 + \kappa_2 r^4 \right)$$

$$y_d = y_n \left(1 + \kappa_1 r^2 + \kappa_2 r^4 \right)$$

3. Apply focal length
translate image center

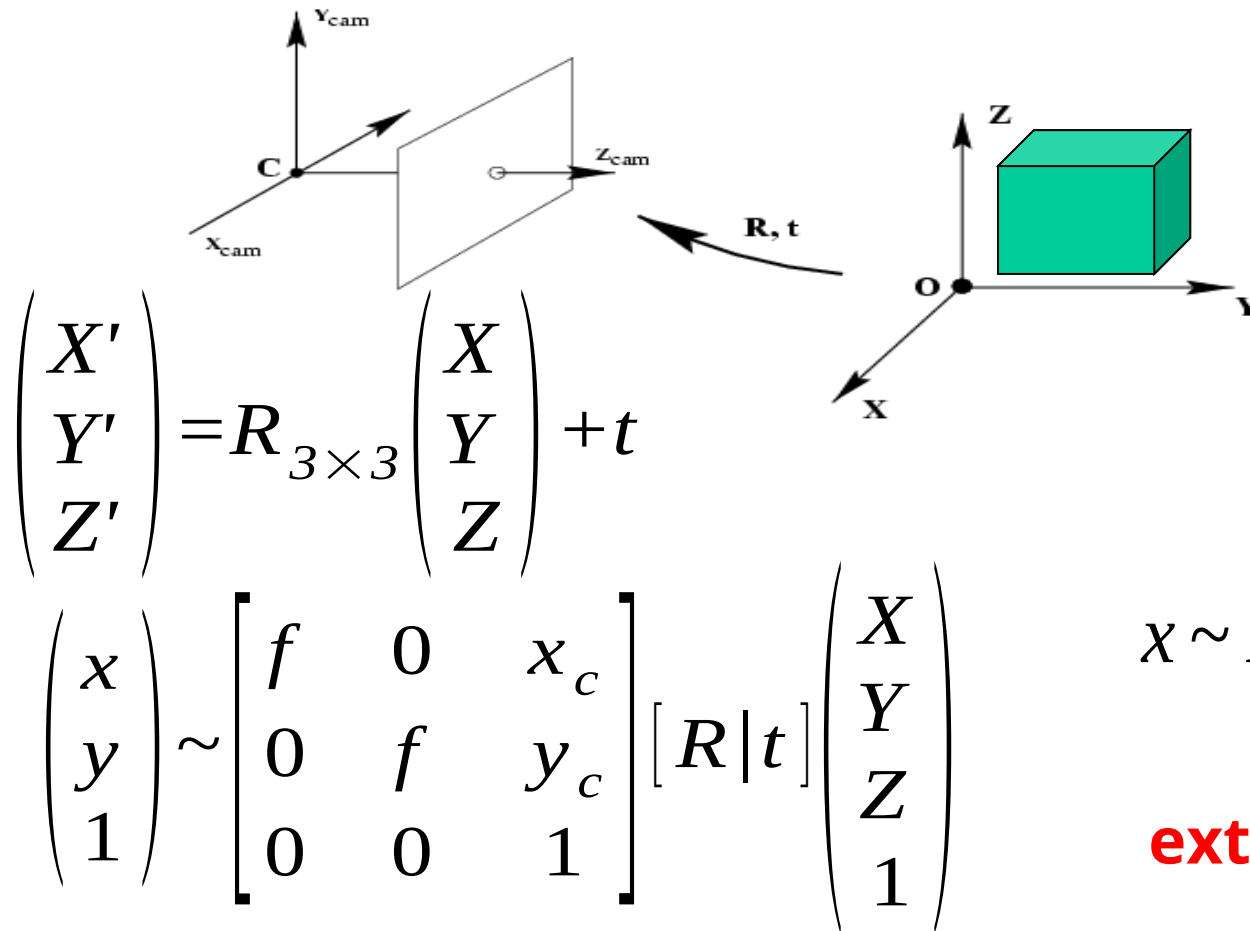
$$x = fx_d + x_c$$

$$y = fy_d + y_c$$

To model lens distortion

Use above projection operation instead of standard projection matrix multiplication

Camera rotation and translation



$$x \sim K \underbrace{[R | t]}_{\text{extrinsic matrix}} X$$

extrinsic matrix

Two kinds of parameters

internal or *intrinsic* parameters:

focal length, optical center, skew

external or *extrinsic* (pose):

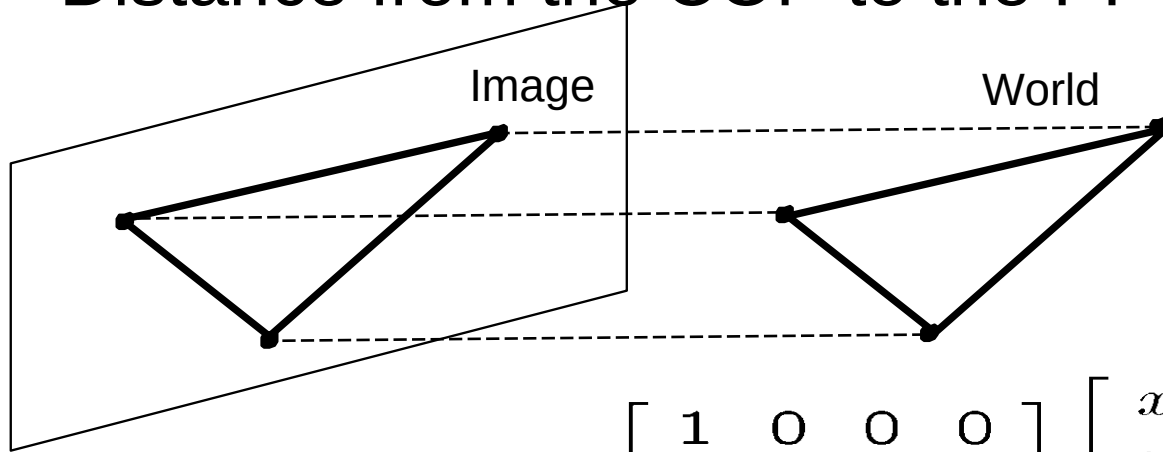
rotation and translation

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \sim \begin{bmatrix} f & 0 & x_c \\ 0 & f & y_c \\ 0 & 0 & 1 \end{bmatrix} [R|t] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} = \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \end{bmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

Orthographic projection

Special case of perspective projection

Distance from the COP to the PP is infinite



$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow (x, y)$$

- Also called “parallel projection”: $(x, y, z) \rightarrow (x, y)$

Other types of projections

Scaled orthographic

Also called “weak perspective”

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1/d \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1/d \end{bmatrix} \Rightarrow (dx, dy)$$

Affine projection

Also called “paraperspective”

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Camera calibration

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \sim \begin{bmatrix} f & 0 & x_c \\ 0 & f & y_c \\ 0 & 0 & 1 \end{bmatrix} [R|t] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} = \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \end{bmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

Estimate both intrinsic and extrinsic parameters

Mainly, two categories:

- Using objects with known geometry as reference
- Self calibration (structure from motion)

Tsai's Algorithm

Roger Tsai, 1987

Estimate

extrinsic parameters (position and orientation of the camera
in the world)

intrinsic parameters (focal length, optical center, and lens
distortion)

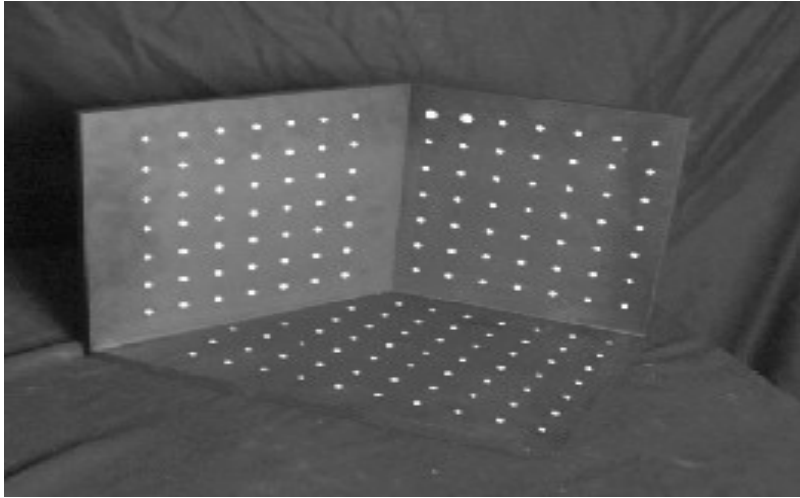
Tsai's Algorithm

Radial Alignment Constraint

Radial distortion (which causes the "barrel" or "pincushion" effect in lenses) affects a pixel's distance from the image center, but it does not alter the angular direction in which that pixel is located relative to the center.

Tsai realized that if he only looked at the vector direction of the points and ignored their radial distance, he could calculate the camera's spatial orientation (the extrinsic parameters) without initially needing to know the lens distortion.

Camera calibration



Directly estimate 11 unknowns in the $\mathbf{M}$ matrix using known 3D points (X_i, Y_i, Z_i) and measured feature positions (u_i, v_i)

$$x \sim K[R|t] X = \mathbf{M}X$$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Linear regression

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$u_i = \frac{m_{00}X_i + m_{01}Y_i + m_{02}Z_i + m_{03}}{m_{20}X_i + m_{21}Y_i + m_{22}Z_i + 1}$$

$$v_i = \frac{m_{10}X_i + m_{11}Y_i + m_{12}Z_i + m_{13}}{m_{20}X_i + m_{21}Y_i + m_{22}Z_i + 1}$$

Linear regression

$$u_i(m_{20}X_i + m_{21}Y_i + m_{22}Z_i + 1) = m_{00}X_i + m_{01}Y_i + m_{02}Z_i + m_{03}$$

$$v_i(m_{20}X_i + m_{21}Y_i + m_{22}Z_i + 1) = m_{10}X_i + m_{11}Y_i + m_{12}Z_i + m_{13}$$

$$\begin{bmatrix} X_i & Y_i & Z_i & 1 & 0 & 0 & 0 & 0 & -u_i X_i & -u_i Y_i & -u_i Z_i \\ 0 & 0 & 0 & 0 & X_i & Y_i & Z_i & 1 & -v_i X_i & -v_i Y_i & -v_i Z_i \end{bmatrix} \begin{bmatrix} m_{00} \\ m_{01} \\ m_{02} \\ m_{03} \\ m_{10} \\ m_{11} \\ m_{12} \\ m_{13} \\ m_{20} \\ m_{21} \\ m_{22} \end{bmatrix} = \begin{bmatrix} u_i \\ v_i \end{bmatrix}$$

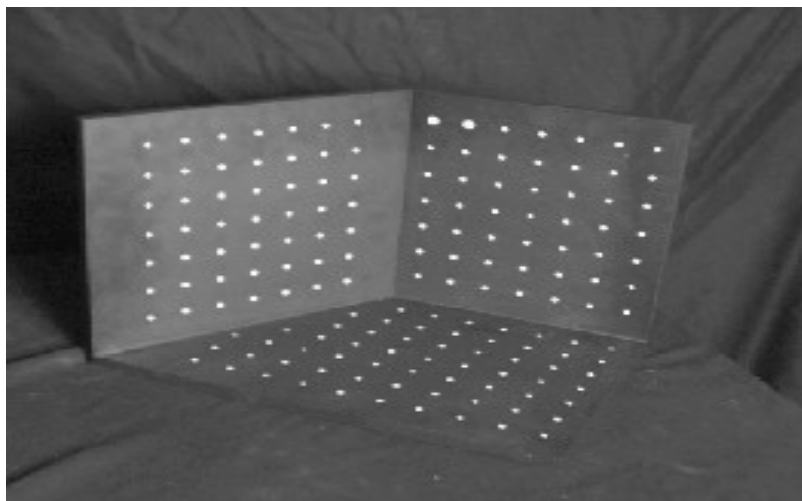
Linear regression with several points

$$\begin{bmatrix}
 X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1 X_1 & -u_1 Y_1 & -u_1 Z_1 \\
 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1 X_1 & -v_1 Y_1 & -v_1 Z_1 \\
 & & & & & & \vdots & & & & \\
 X_N & Y_N & Z_N & 1 & 0 & 0 & 0 & 0 & -u_N X_N & -u_N Y_N & -u_N Z_N \\
 0 & 0 & 0 & 0 & X_N & Y_N & Z_N & 1 & -v_N X_N & -v_N Y_N & -v_N Z_N
 \end{bmatrix}
 \begin{bmatrix}
 m_{00} \\
 m_{01} \\
 m_{02} \\
 m_{03} \\
 m_{10} \\
 m_{11} \\
 m_{12} \\
 m_{13} \\
 m_{20} \\
 m_{21} \\
 m_{22}
 \end{bmatrix}
 =
 \begin{bmatrix}
 u_i \\
 v_i
 \end{bmatrix}$$

Solve for Projection Matrix M using least-square techniques

At least 11 points

Non Linear Calibration



Why Non-Linear?

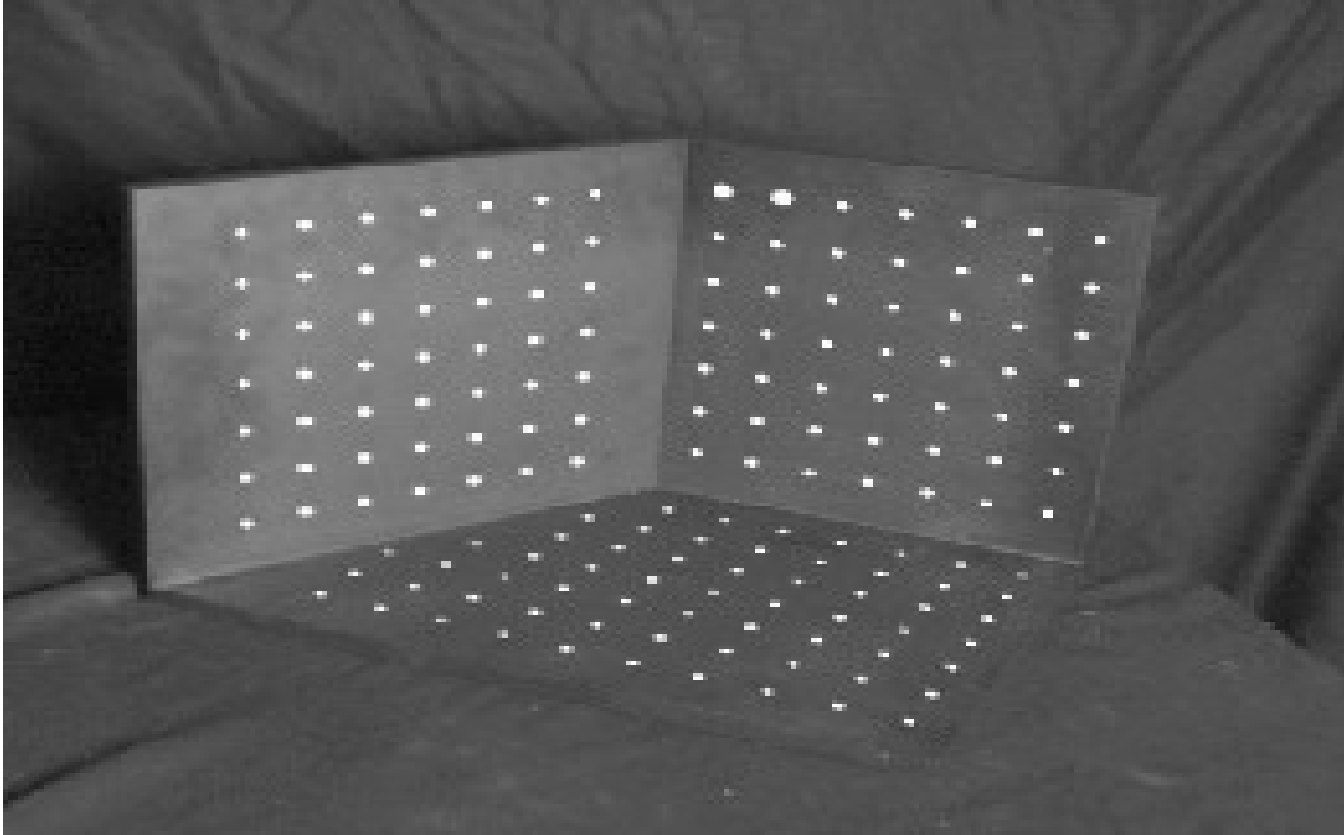
Linear approach minimizes algebraic error and ignores lens distortion

Non-linear approach minimizes the true Reprojection Error (geometric distance in pixels) and includes distortion parameters

$$x \sim K \begin{bmatrix} R & t \end{bmatrix} X = \mathbf{M}X$$

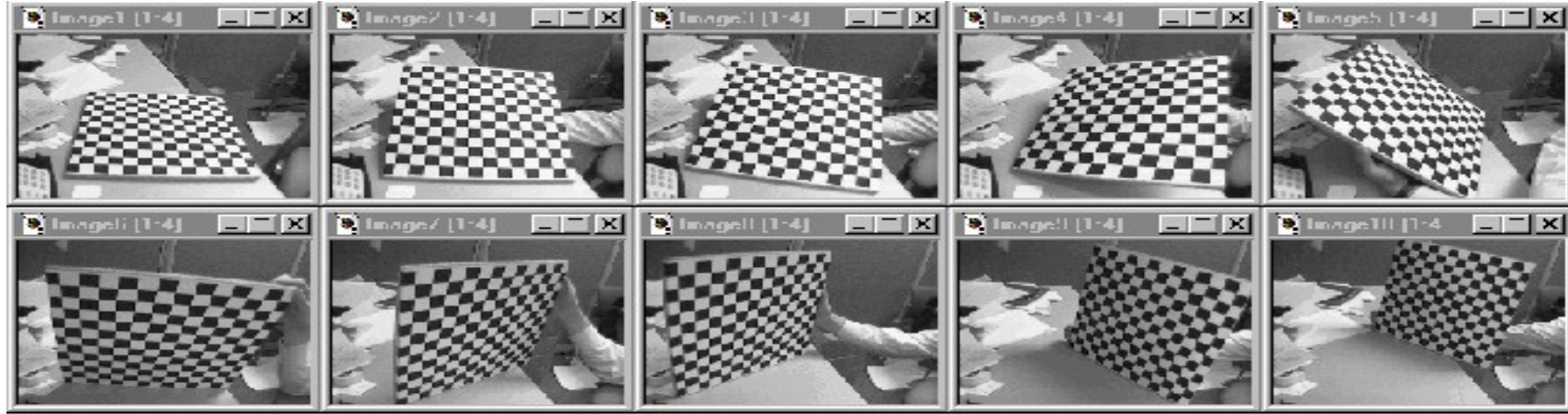
$$\sum_{i=1}^N \left(u_i - \frac{m_{00}X_i + m_{01}Y_i + m_{02}Z_i + m_{03}}{m_{20}X_i + m_{21}Y_i + m_{22}Z_i + 1} \right)^2 + \left(v_i - \frac{m_{10}X_i + m_{11}Y_i + m_{12}Z_i + m_{13}}{m_{20}X_i + m_{21}Y_i + m_{22}Z_i + 1} \right)^2$$

Practical Issue



It is hard to make the pattern and 3D feature positions are difficult to measure!

Zhang's 2000 Multi-plane calibration



Jean-Yves Bouguet, Intel Corp.

Advantage

- Only requires a plane

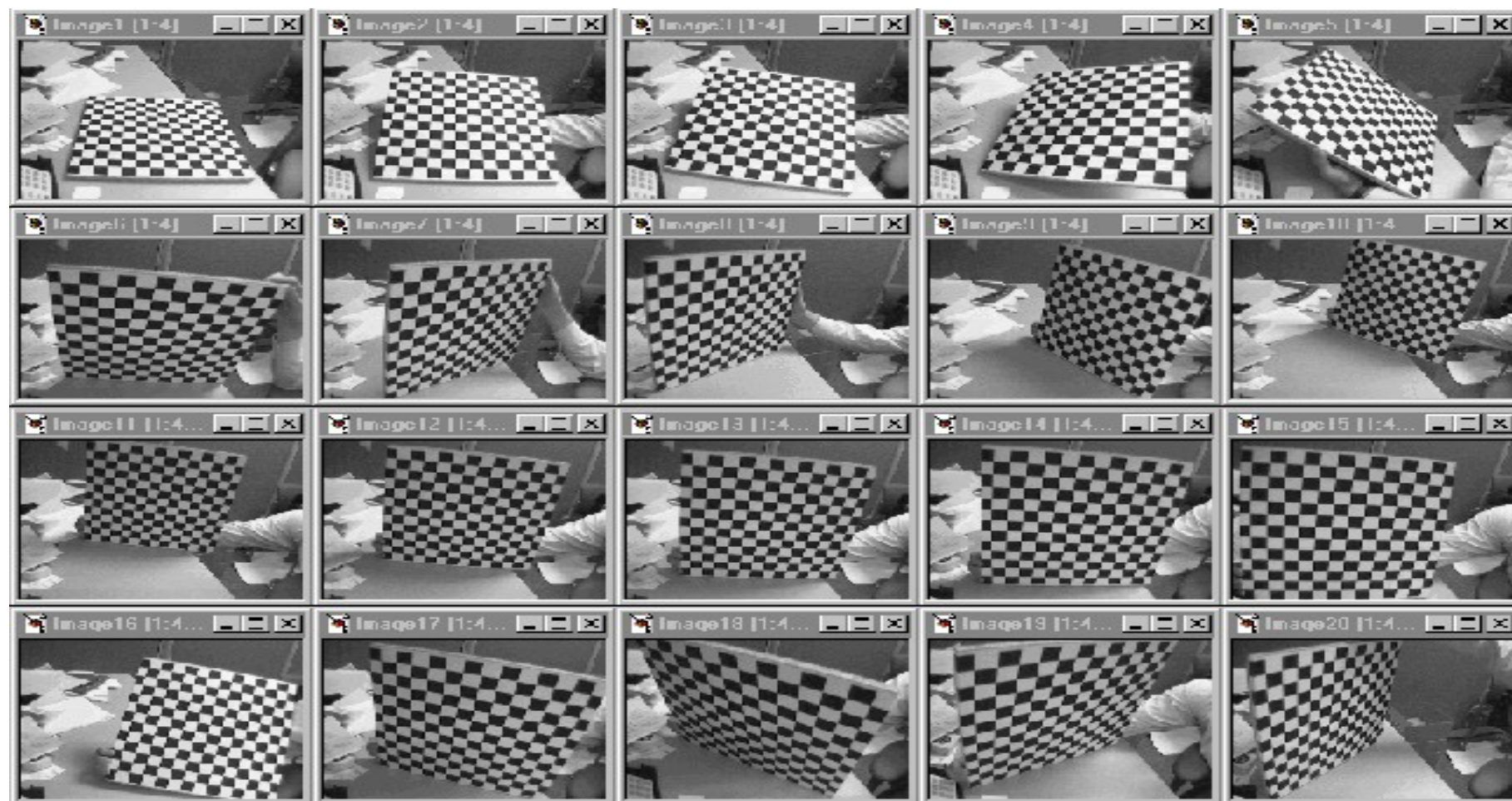
- Don't have to know positions/orientations

- Code available

Intel's OpenCV library: <http://www.intel.com/research/mrl/research/opencv/>

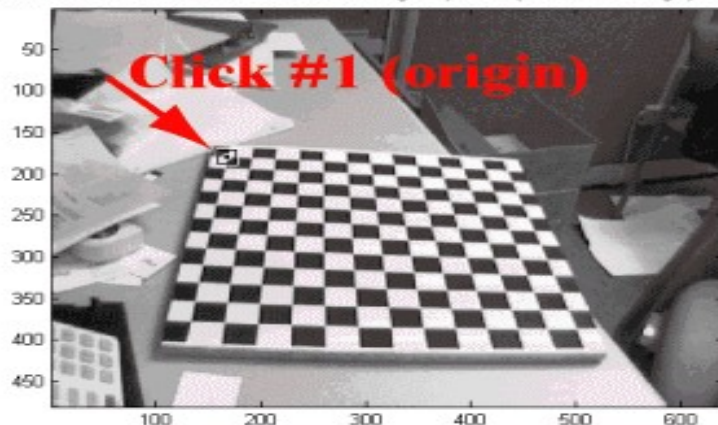
Zhengyou Zhang's web site: <http://research.microsoft.com/~zhang/Calib/>

Step 1: data acquisition

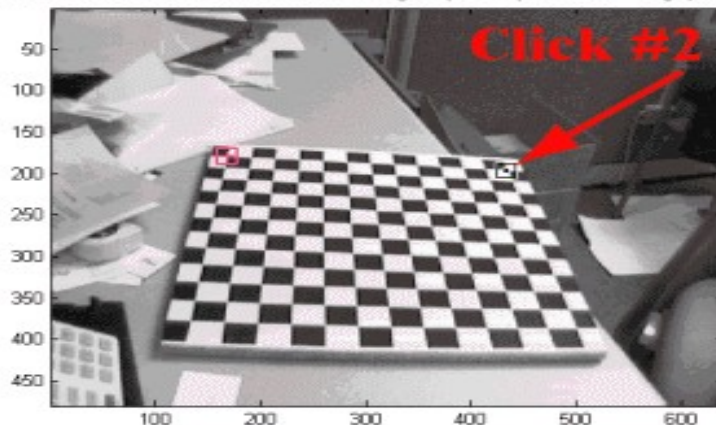


Step 2: specify corner order

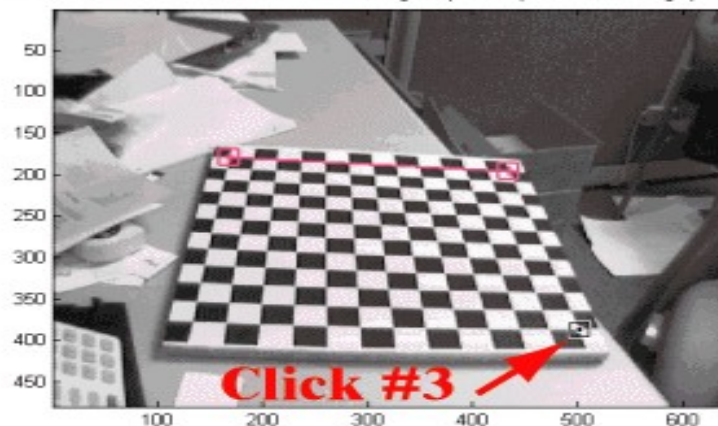
Click on the four extreme corners of the rectangular pattern (first corner = origin)... Image 1



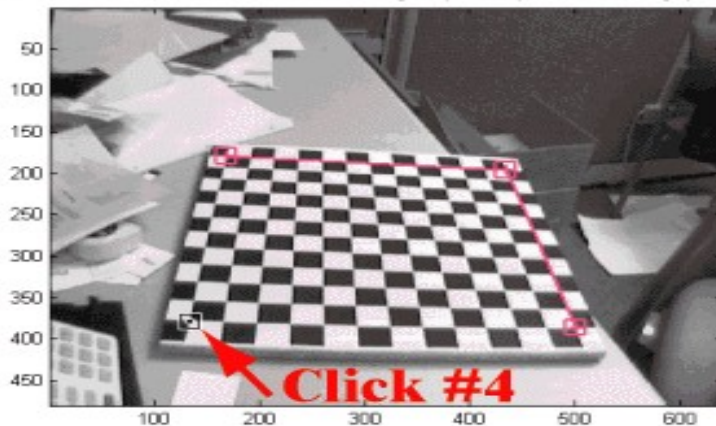
Click on the four extreme corners of the rectangular pattern (first corner = origin)... Image 1



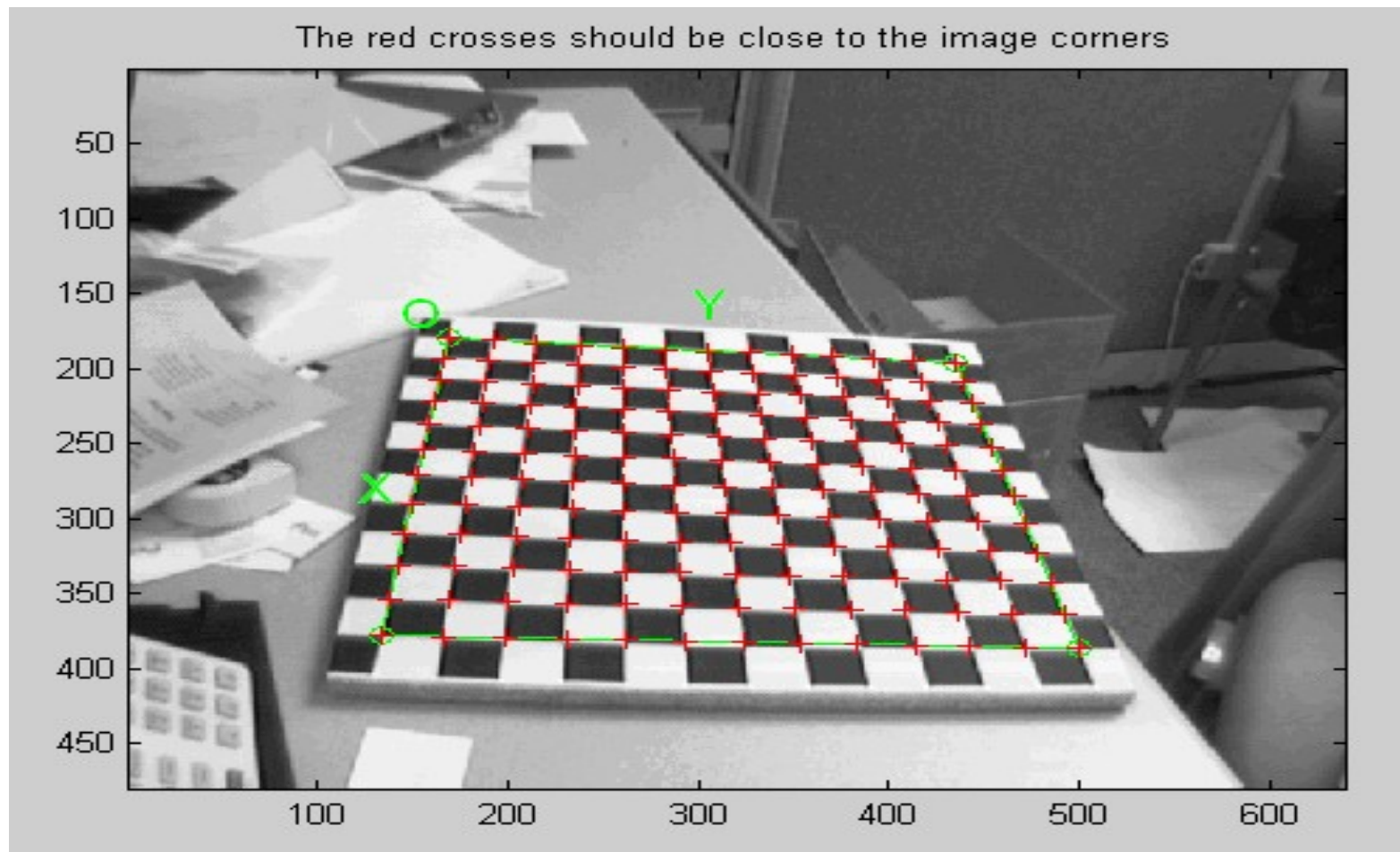
Click on the four extreme corners of the rectangular pattern (first corner = origin)... Image 1



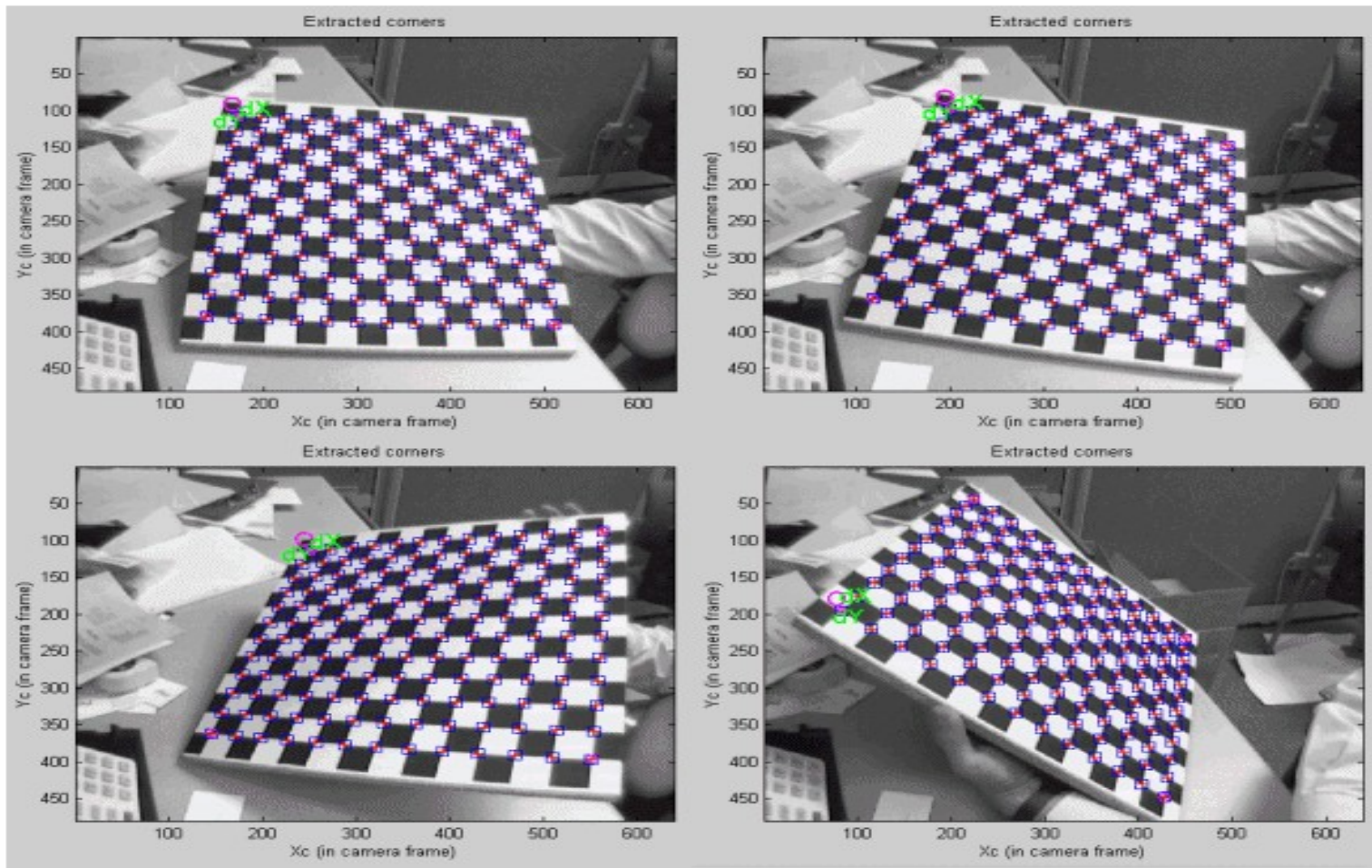
Click on the four extreme corners of the rectangular pattern (first corner = origin)... Image 1



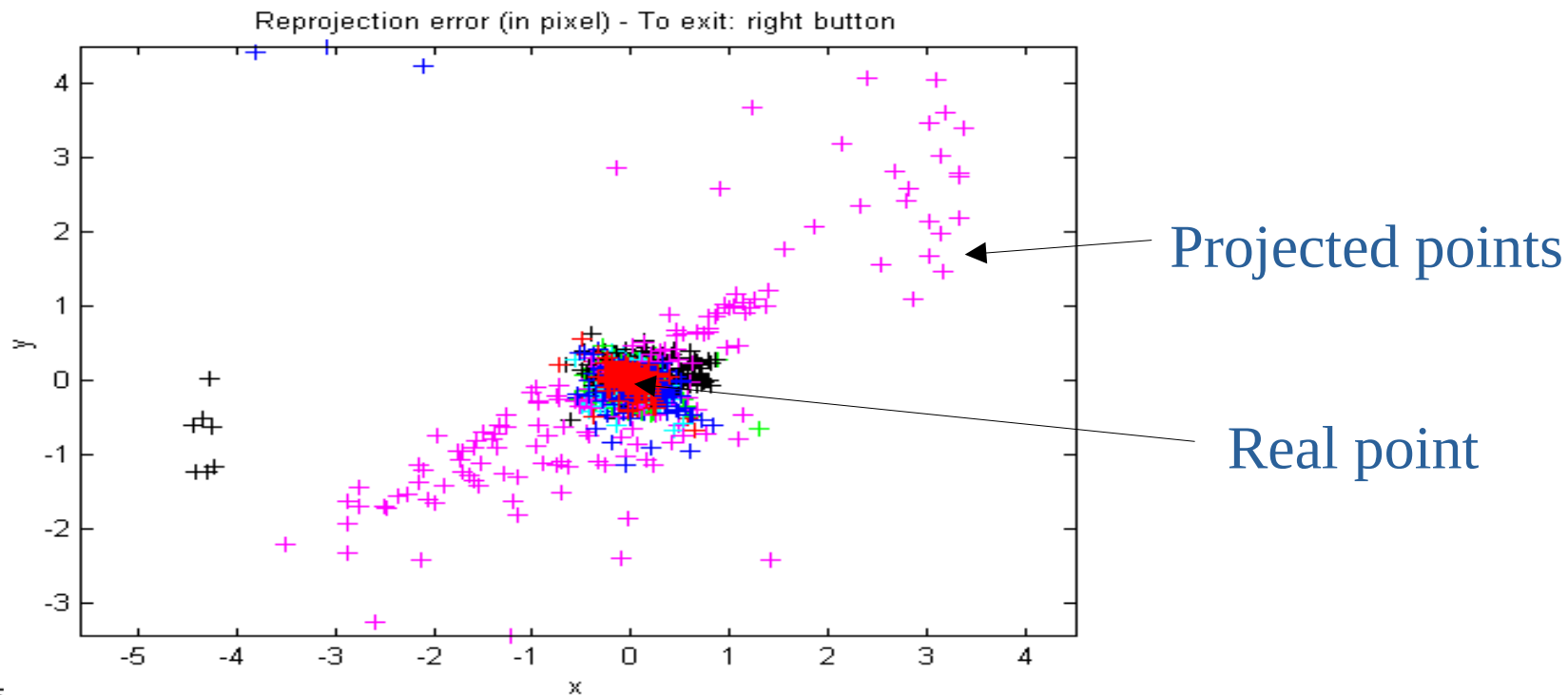
Step 3: corner extraction



Step 3: corner extraction



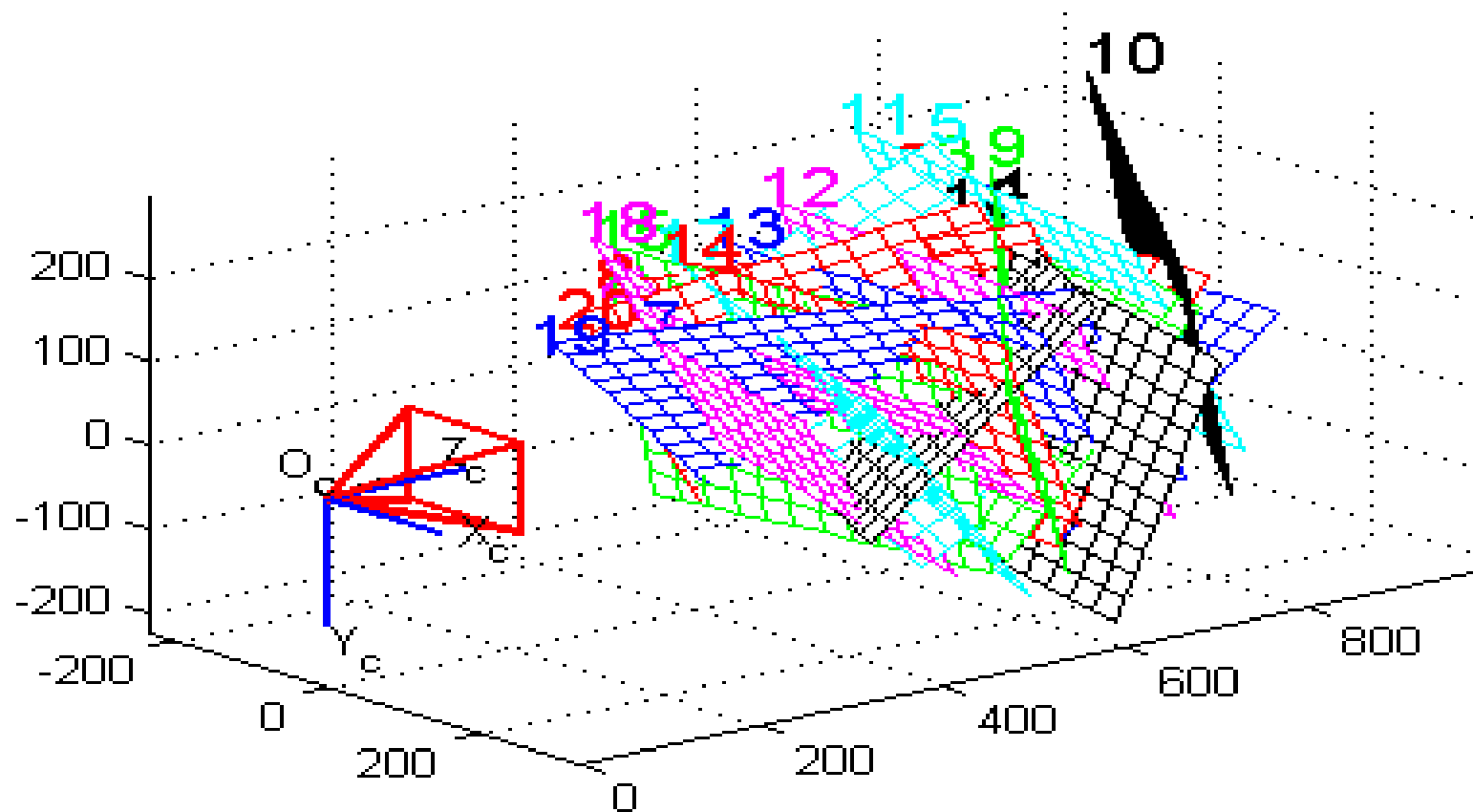
Step 4: minimize reprojection error



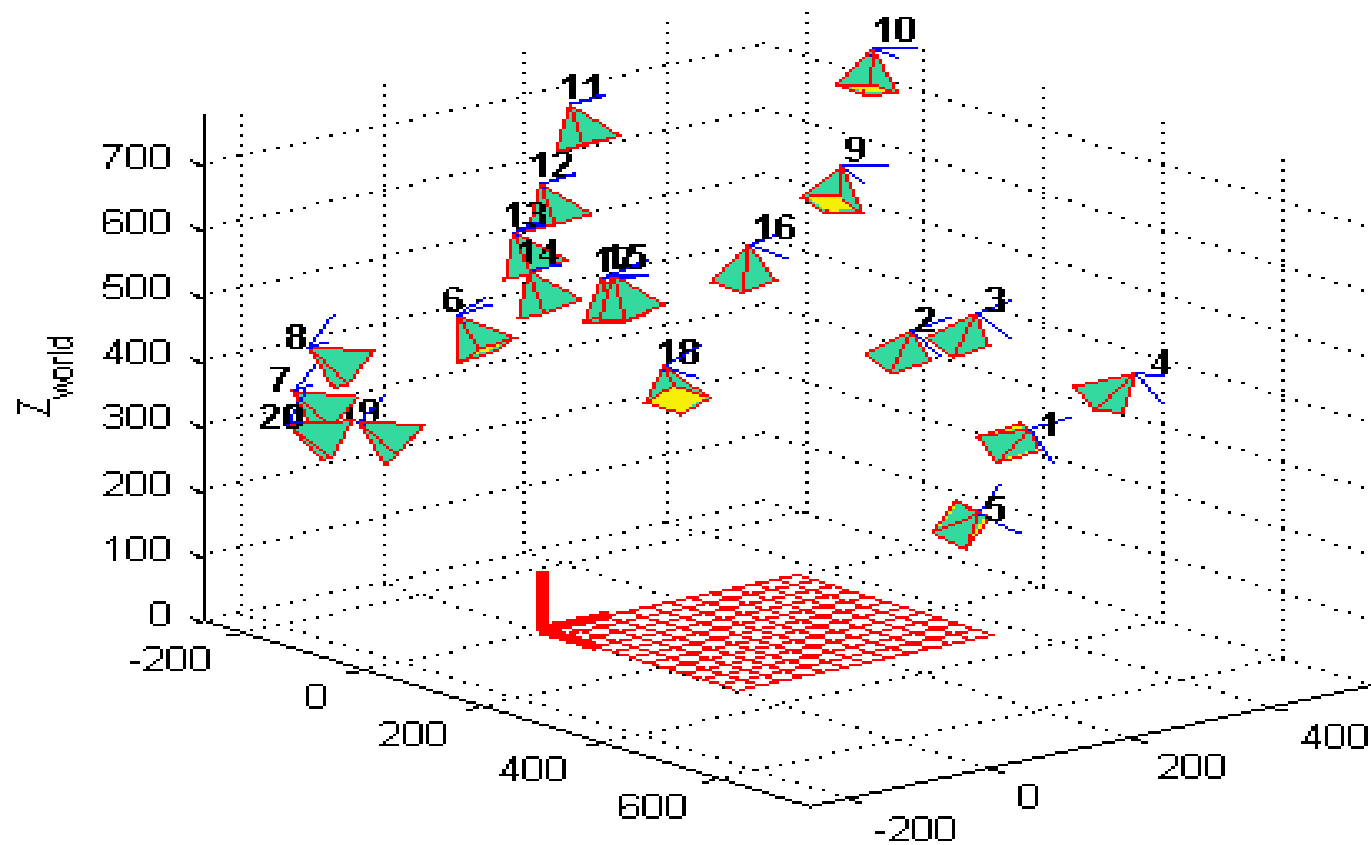
Calibration res

```
Focal Length:      fc = [ 657.46290  657.94673 ] ± [ 0.31819  0.34046 ]
Principal point:    cc = [ 303.13665  242.56935 ] ± [ 0.64682  0.59218 ]
Skew:              alpha_c = [ 0.00000 ] ± [ 0.00000 ] => angle of pixel axes =
Distortion:         kc = [ -0.25403  0.12143  -0.00021  0.00002  0.00000 ]
Pixel error:        err = [ 0.11689  0.11500 ]
```

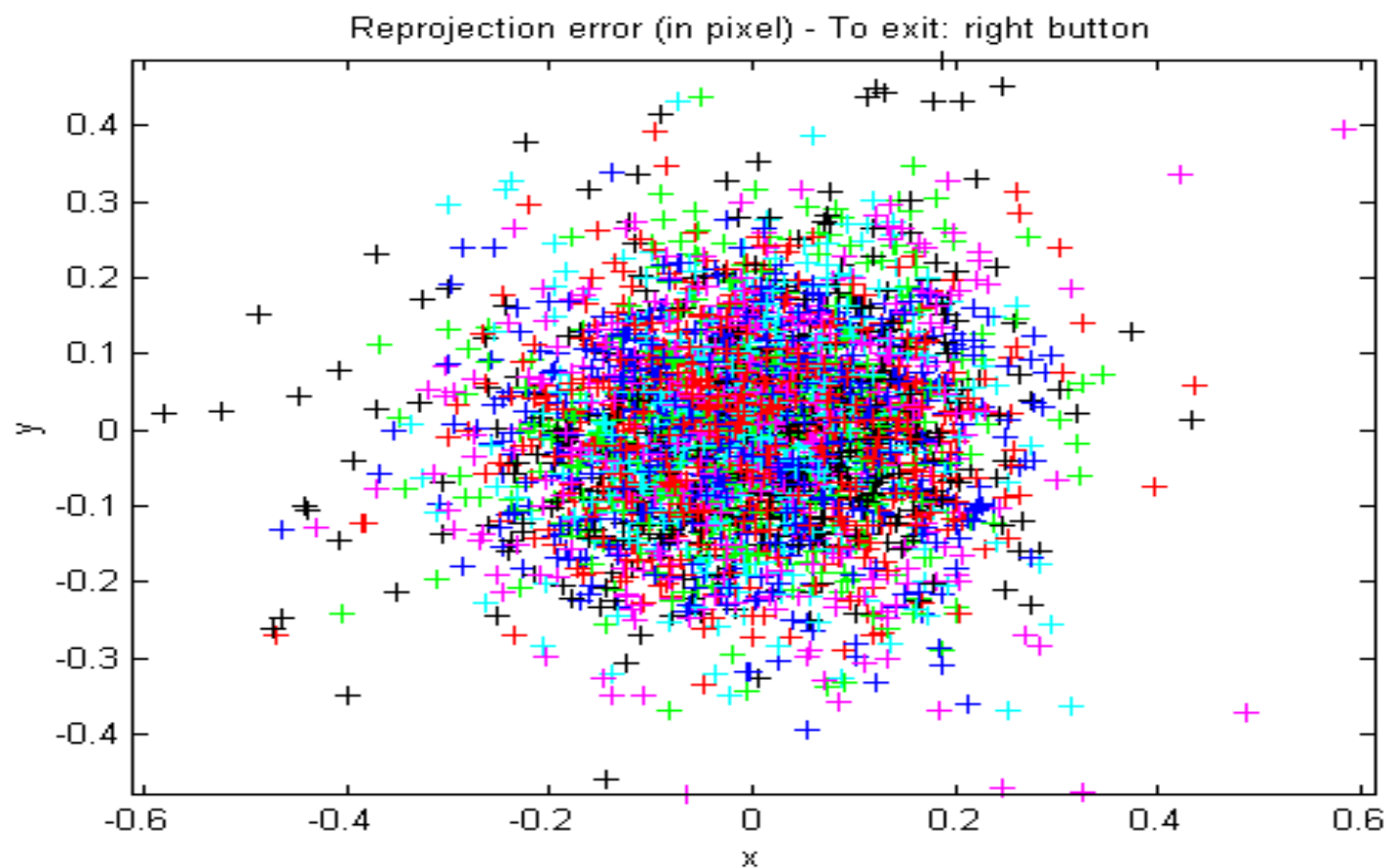
Step 4: camera calibration



Step 4: camera calibration



Step 5: refinement



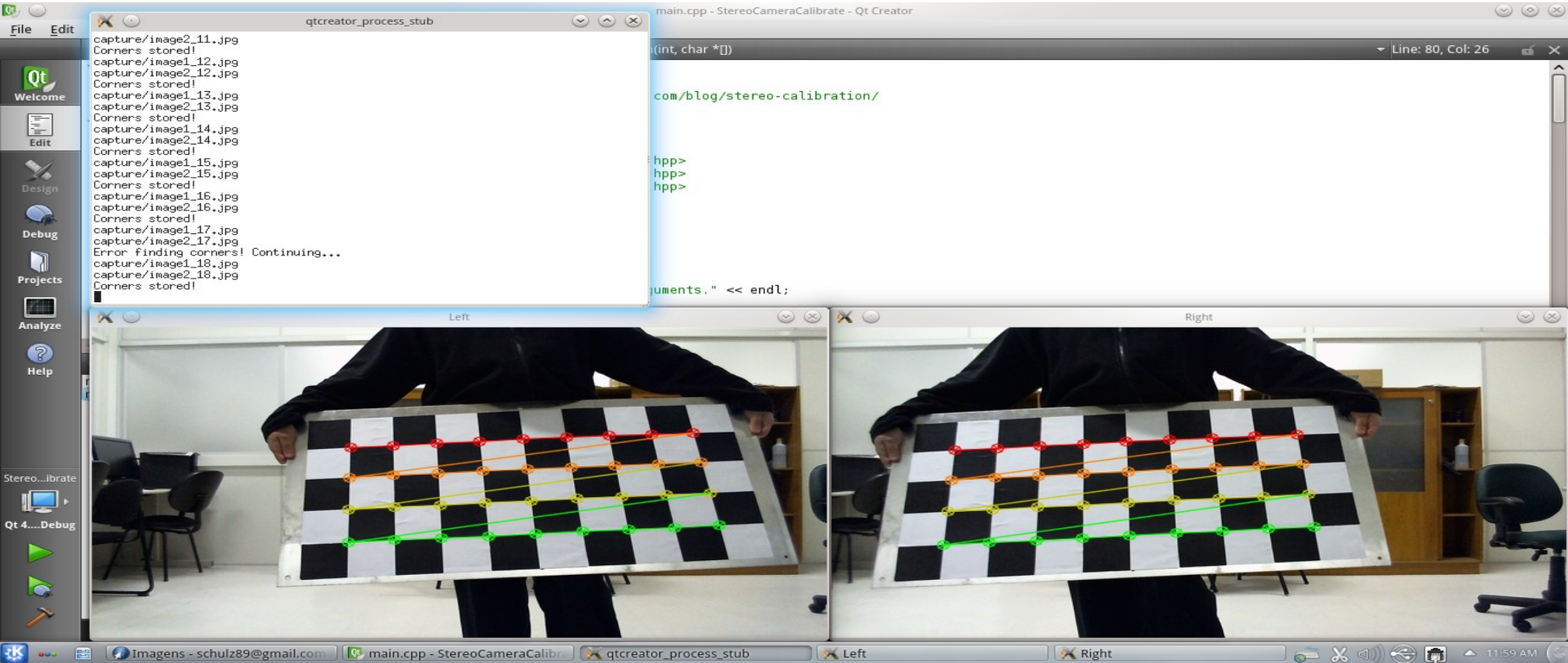
Camera calibration With OpenCV

[http://docs.opencv.org/master/doc/tutorials/calib3d/
camera_calibration/camera_calibration.html](http://docs.opencv.org/master/doc/tutorials/calib3d/camera_calibration/camera_calibration.html)

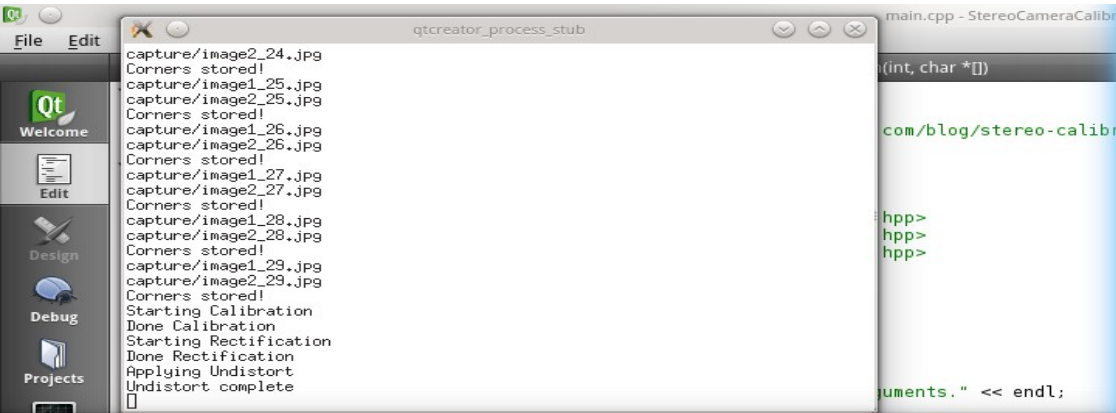
Calibração de Câmeras utilizando OpenCV stereoCalibration

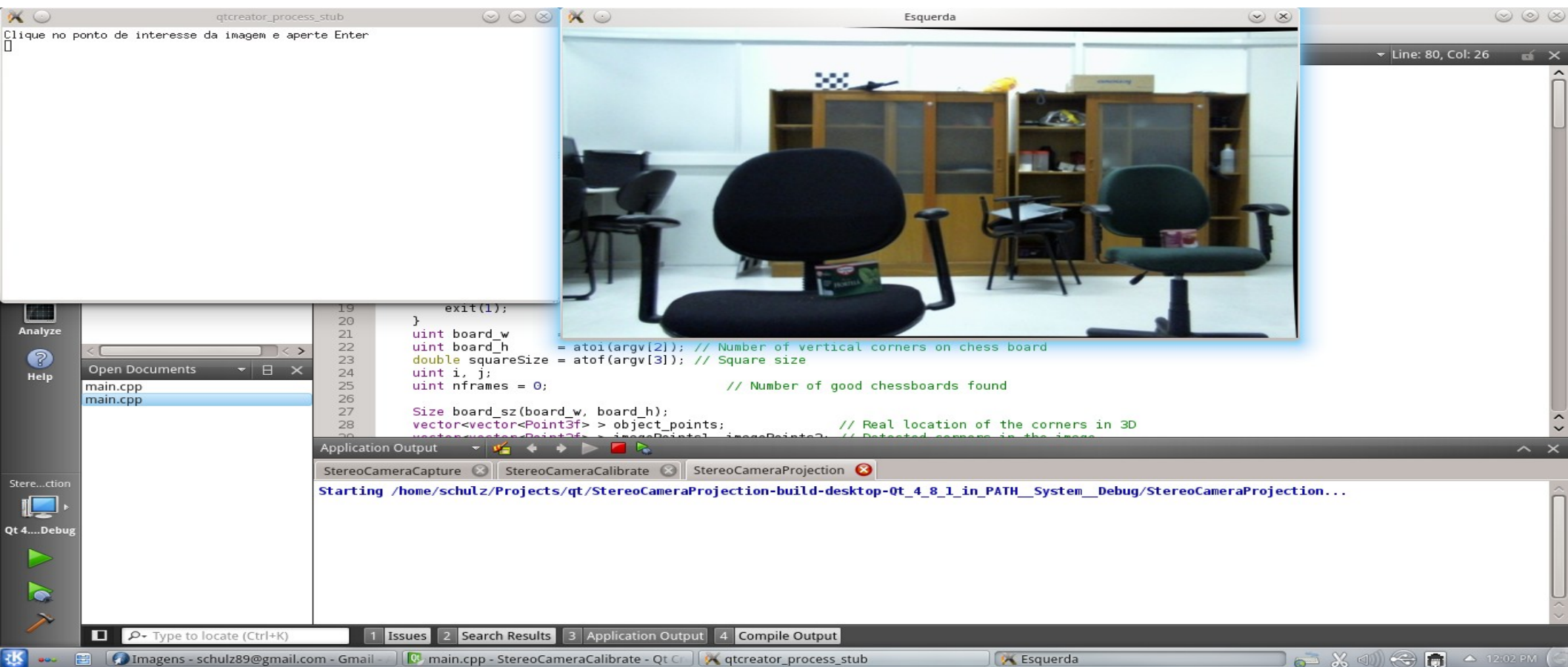
Victor Schulz

VRI

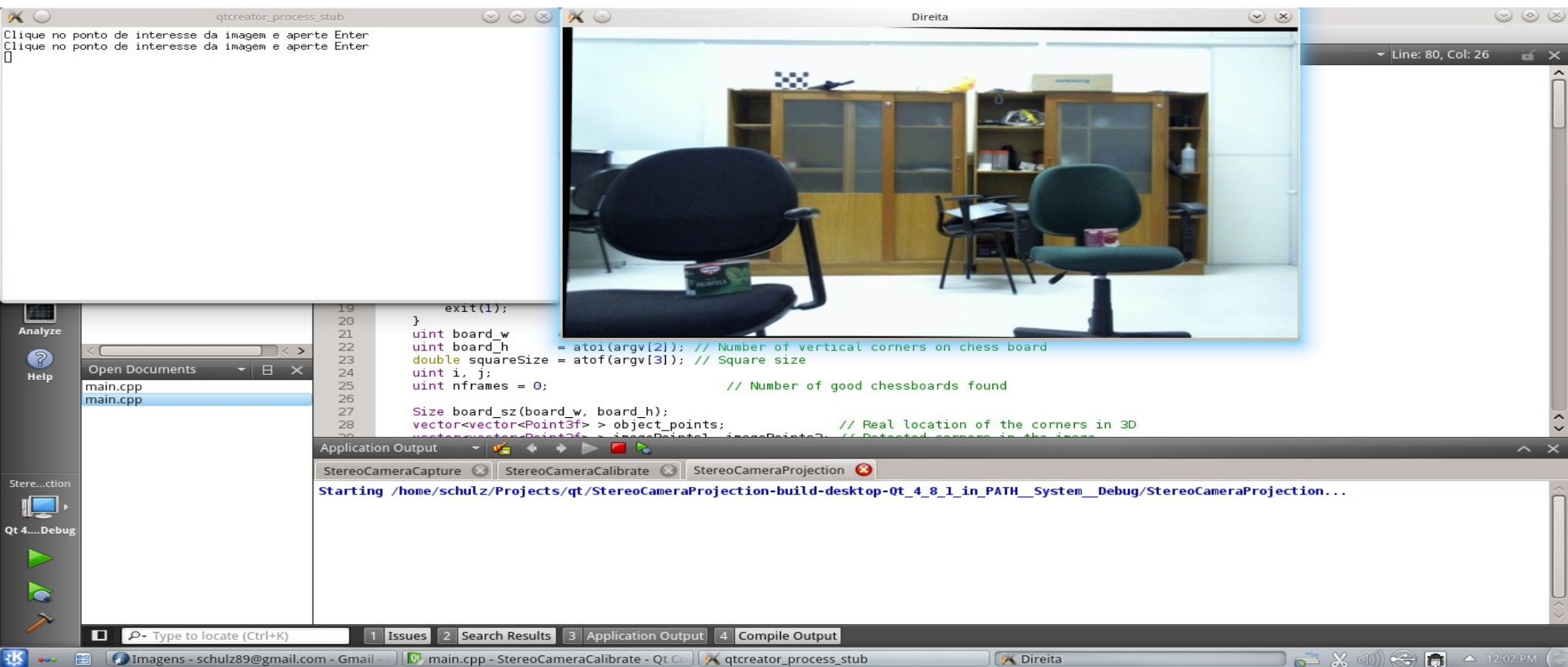


Tamanho informado do tabuleiro: 4x9 (sem as bordas)





Caixa de chá a aprox. 1,3m



Caixa de chá a aprox. 2,2m

qtcreator_process_stub

Clique no ponto de interesse da imagem e aperte Enter
Clique no ponto de interesse da imagem e aperte Enter
X: 24.0253
Y: -22.0424
Z: -125.089
Norma: 129.269
Press <RETURN> to close this window...

main.cpp - StereoCameraCalibrate - Qt Creator

Line: 80, Col: 26

```
main(int, char *[])  
  
ation  
  
/jayrambhia.com/blog/stereo-calibration/  
  
3d/calib3d.hpp>  
gui/highgui.hpp>  
oc/imgproc.hpp>  
  
* argv[]  
  
reading arguments." << endl;
```

Analyze
Help

Open Documents

- main.cpp
- main.cpp

```
19     exit(1);  
20 }  
21 uint board_w = atoi(argv[1]); // Number of horizontal corners on chess board  
22 uint board_h = atoi(argv[2]); // Number of vertical corners on chess board  
23 double squareSize = atof(argv[3]); // Square size  
24 uint i, j;  
25 uint nframes = 0; // Number of good chessboards found  
26  
27 Size board_sz(board_w, board_h);  
28 vector<vector<Point3f> > object_points; // Real location of the corners in 3D  
29 vector<vector<Point2f> > imagePoints2; // Detected corners in the image
```

Stereo...ction
Qt 4....Debug

Application Output

StereoCameraCapture StereoCameraCalibrate StereoCameraProjection

Starting /home/schulz/Projects/qt/StereoCameraProjection-build-desktop-Qt_4_8_1_in_PATH_System_Debug/StereoCameraProjection...
/home/schulz/Projects/qt/StereoCameraProjection-build-desktop-Qt_4_8_1_in_PATH_System_Debug/StereoCameraProjection exited with code 0

Type to locate (Ctrl+K)

1 Issues 2 Search Results 3 Application Output 4 Compile Output

Imagens - schulz89@gmail.com - Gmail - Abrow main.cpp - StereoCameraCalibrate - Qt Creator qtcreator_process_stub

12:02 PM

qtcreator_process_stub

Clique no ponto de interesse da imagem e aperte Enter
Clique no ponto de interesse da imagem e aperte Enter
X: -63.2872
Y: -22.0462
Z: -212.108
Norma: 222.443
Press <RETURN> to close this window...

main.cpp - StereoCameraCalibrate - Qt Creator

Line: 80, Col: 26

```
main(int, char *[])  
  
ration  
  
/jayrambhia.com/blog/stereo-calibration/  
  
>3d/calib3d.hpp>  
>gui/highgui.hpp>  
>roc/imgproc.hpp>  
  
* argv[])  
  
reading arguments." << endl;  
  
19     exit(1);  
20 }  
21 uint board_w    = atoi(argv[1]); // Number of horizontal corners on chess board  
22 uint board_h    = atoi(argv[2]); // Number of vertical corners on chess board  
23 double squareSize = atof(argv[3]); // Square size  
24 uint i, j;  
25 uint nframes = 0;                // Number of good chessboards found  
26  
27 Size board_sz(board_w, board_h);  
28 vector<vector<Point3f> > object_points; // Real location of the corners in 3D  
29 vector<vector<Point3f> > imagePoints2; // Detected corners in the image
```

Analyze
Help

Open Documents

main.cpp
main.cpp

Stere...ction
Qt 4....Debug

Application Output

StereoCameraCapture StereoCameraCalibrate StereoCameraProjection

Starting /home/schulz/Projects/qt/StereoCameraProjection-build-desktop-Qt_4_8_1_in_PATH_System_Debug/StereoCameraProjection...
/home/schulz/Projects/qt/StereoCameraProjection-build-desktop-Qt_4_8_1_in_PATH_System_Debug/StereoCameraProjection exited with code 0

Starting /home/schulz/Projects/qt/StereoCameraProjection-build-desktop-Qt_4_8_1_in_PATH_System_Debug/StereoCameraProjection...
/home/schulz/Projects/qt/StereoCameraProjection-build-desktop-Qt_4_8_1_in_PATH_System_Debug/StereoCameraProjection exited with code 0

Type to locate (Ctrl+K)

1 Issues 2 Search Results 3 Application Output 4 Compile Output

Imagens - schulz89@gmail.com - Gmail - Abro... main.cpp - StereoCameraCalibrate - Qt Creator qtcreator_process_stub

12:03 PM

Com as câmeras com eixos principais não paralelos



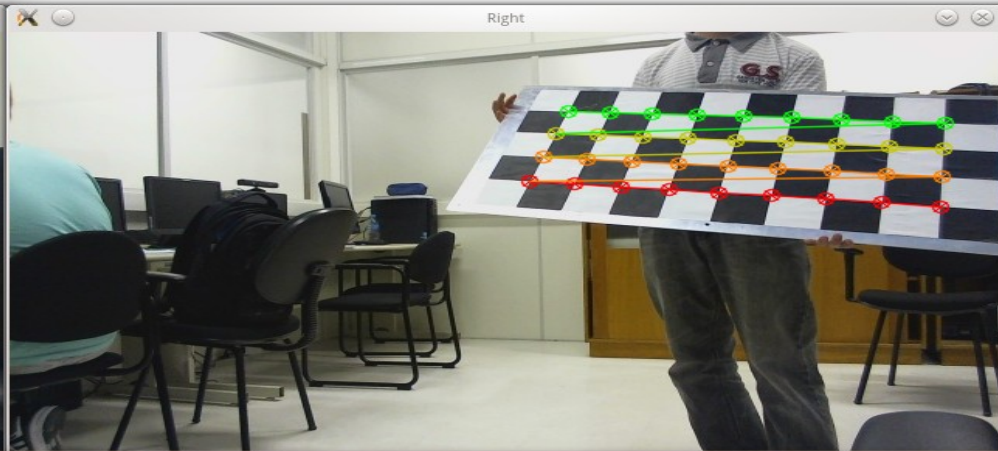
qtcreator_process_stub

Reading imglist.txt...
capture/image1_0.jpg
capture/image2_0.jpg
Corners stored!
capture/image1_1.jpg
capture/image2_1.jpg
Corners stored!
capture/image1_2.jpg
capture/image2_2.jpg
Corners stored!
capture/image1_3.jpg
capture/image2_3.jpg
Corners stored!
capture/image1_4.jpg
capture/image2_4.jpg
Corners stored!
capture/image1_5.jpg
capture/image2_5.jpg
Corners stored!
capture/image1_6.jpg
capture/image2_6.jpg
Corners stored!

main.cpp - StereoCameraCalibrate - Qt Creator

Line: 80, Col: 25

```
main(int, char *[])  
  
    cornerSubPix(gray1, corners1, Size(11, 11), Size(-1, -1), TermCriteria(CV_TERMCRIT_EPS | CV_TERMCRIT_ITER, 30, 0.1));  
    drawChessboardCorners(img1, board_sz, corners1, found1);  
    cornerSubPix(gray2, corners2, Size(11, 11), Size(-1, -1), TermCriteria(CV_TERMCRIT_EPS | CV_TERMCRIT_ITER, 30, 0.1));  
    drawChessboardCorners(img2, board_sz, corners2, found2);  
  
    imagePoints1.push_back(corners1);  
    imagePoints2.push_back(corners2);  
    objectPoints.push_back(obj);  
    cout << "Corners stored!" << endl;  
    frames++;  
  
    cout << "Error finding corners! Continuing..." << endl;  
  
    imshow("Left", img1);  
    imshow("Right", img2);  
    wait(0);
```

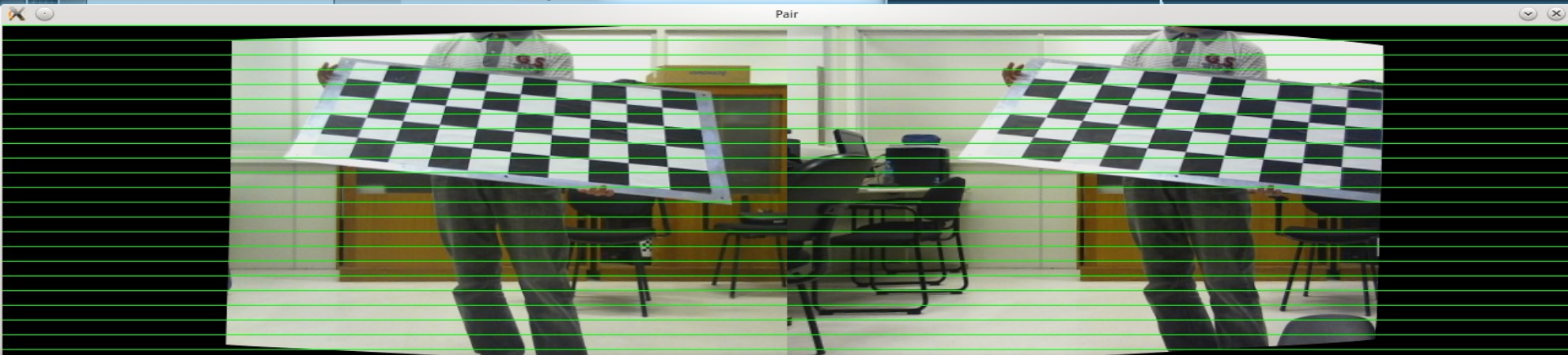


qtcreator_process_stub

```
capture/image2_9.jpg  
Corners stored!  
capture/image1_10.jpg  
capture/image2_10.jpg  
Corners stored!  
capture/image1_11.jpg  
capture/image2_11.jpg  
Corners stored!  
capture/image1_12.jpg  
capture/image2_12.jpg  
Corners stored!  
capture/image1_13.jpg  
capture/image2_13.jpg  
Corners stored!  
capture/image1_14.jpg  
capture/image2_14.jpg  
Corners stored!  
Starting Calibration  
Done Calibration  
Starting Rectification  
Done Rectification  
Applying Undistort  
Undistort complete  
□
```

main.cpp - StereoCameraCalib

```
main(int, char *[])  
  
    cornerSubPix(gray1, corners1, Size(1  
    drawChessboardCorners(img1, board_sz  
    cornerSubPix(gray2, corners2, Size(1  
    drawChessboardCorners(img2, board_sz  
  
    imagePoints1.push_back(corners1);  
    imagePoints2.push_back(corners2);  
    object_points.push_back(obj);  
    cout << "Corners stored!" << endl;  
    frames++;  
  
    cout << "Error finding corners! Cont  
  
    ("Left",img1);  
    ("Right",img2);  
    (0);
```



OpenCV stereoCalibration

CM1 - Camera Matrix for left camera

CM2 - Camera Matrix for right camera

D1 - Vector of distortion coefficients for left camera

D2 - Vector of distortion coefficients for right camera

R - Rotation matrix between the left and the right camera coordinate systems

T - Translation vector between the left and the right coordinate systems of the cameras

E - Essential matrix

F - Fundamental matrix

Computational Vision and Perception

Prof. Eduardo Todt

todt@ufpr.br

